
🔐 IAM Policies – Demo Guide

🎯 Objective

You will:

- Create a **custom IAM policy**
- Attach it to a **user**
- Test permissions with **AWS CLI**
- Learn to **analyze deny/allow results**

📌 Step 1: Create a Custom IAM Policy

Let's create a policy that **allows full access to a specific S3 bucket**.

✅ Sample Policy: S3 Full Access to One Bucket

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::your-bucket-name",
        "arn:aws:s3:::your-bucket-name/*"
      ]
    }
  ]
}
```

> Replace `your-bucket-name` with an actual bucket name.

🛠️ Create Policy via Console

1. Go to **IAM** → **Policies** → **Create Policy**
2. Select **JSON** tab and paste the above JSON
3. Click **Next**, name it e.g. `S3BucketFullAccessPolicy`
4. Click **Create Policy**

🛠️ Create Policy via CLI

```
aws iam create-policy \
  --policy-name S3BucketFullAccessPolicy \
  --policy-document file://s3-policy.json
```

👤 Step 2: Create a User and Attach the Policy

```
aws iam create-user --user-name devuser

aws iam attach-user-policy \
  --user-name devuser \
  --policy-arn arn:aws:iam::123456789012:policy/S3BucketFullAccessPolicy
```

You can also attach to a **group** instead.

🔐 Step 3: Generate Access Keys for CLI Access

```
aws iam create-access-key --user-name devuser
```

Save the `AccessKeyId` and `SecretAccessKey`.

💻 Step 4: Test Permissions Using AWS CLI

👉 Configure CLI with devuser credentials:

aws configure --profile devuser

Enter access key, secret key, region, and output format.

👉 Try allowed and restricted actions:

```
# Should succeed
aws s3 ls s3://your-bucket-name --profile devuser
aws s3 cp file.txt s3://your-bucket-name/ --profile devuser

# Should fail (access to another bucket not granted)
aws s3 ls s3://some-other-bucket --profile devuser
```

🧠 Step 5: Evaluate IAM Policy Behavior

If something fails:

- Use the **IAM Policy Simulator** to test
- Check for any **Service Control Policies (SCPs)**, **explicit denies**, or **missing resources**

🗒️ Summary Notes

Term	Meaning
Policy	JSON document that defines `Allow`/`Deny` for actions on resources
Managed Policy	Reusable, attachable policy (AWS or custom)
Inline Policy	Embedded in a single user/group/role
Action	AWS operations like `s3:GetObject`, `ec2:StartInstances`
Resource	ARN of AWS resources (e.g., S3 buckets, EC2 instances)
Effect	Can be `Allow` or `Deny`

🔐 Best Practices

- Use **least privilege**: grant only the permissions needed
- Prefer **managed policies** over inline ones
- Use **policy conditions** to restrict access (e.g., IP, MFA)
